МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу “Компьютерная графика”

Камера и освещение

Выполнил: Д. И. Шнайдер
Группа: М8О-308БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: реализовать «камеру» с возможностью её перемещения и освещение по модели Блинна-Фонга с использованием нескольких типов источников света. Программа должна работать в реальном времени, использовать Vulkan, GLFW и ImGui и позволять через пользовательский интерфейс менять параметры камеры (позиция, поворот, угол обзора), источников света (позиция, цвет и интенсивность, а также параметры, уникальные для каждого типа источника) и материалов объектов (альбедо, цвет блика, коэффициент блика). У вершин, помимо позиции, должен быть задан атрибут нормали, нормали генерируются процедурно.

Вариант 3 - параллелепипед. Номер варианта: ((27 − 1) mod 12) + 1 = 3. Используется объект из лабораторной работы №1.

Выполнены базовое задание и все четыре дополнительных задания. Базовое задание: отрисовка объекта относительно камеры с единственным направленным источником света; все параметры камеры, источника, материала по модели Блинна-Фонга и рассеянный (фоновый) свет изменяются из интерфейса ImGui. Дополнительные задания:

1. разделение внешних данных на два набора дескрипторов: набор сцены (uniform-буфер с параметрами сцены и общими для всех объектов данными, storage-буферы с массивами источников) и набор модели (матрица трансформации и параметры материала);
2. освещение по модели Блинна-Фонга массивом точечных источников из storage-буфера с затуханием по закону обратных квадратов; добавление и удаление источников и изменение их параметров в интерфейсе;
3. освещение по модели Блинна-Фонга массивом прожекторов из storage-буфера с затуханием по закону обратных квадратов и мягкими краями освещаемой области, если внешний угол больше внутреннего; добавление и удаление прожекторов и изменение их параметров в интерфейсе;
4. визуализация источников освещения с возможностью показа и скрытия в интерфейсе: кубик цвета источника для точечных источников и прожекторов, для прожектора дополнительно направление и угол (конус из белых линий); фигуры визуализации имеют постоянный цвет и не освещаются по модели Блинна-Фонга.

Метод решения

Работа выполнена на основе кода лабораторной работы №1, построенного на стартовом проекте курса (Vulkan, GLFW, ImGui, vk-bootstrap, Vulkan Memory Allocator, сборка CMake). Код работы находится в файлах `application.hpp` и `application.cpp`; векторы и матрицы взяты из собственного заголовочного файла `math3d.hpp` первой работы; шейдеры объектов `blinn_phong.vert`, `blinn_phong.frag` и шейдеры визуализации источников `light_marker.vert`, `light_marker.frag` компилируются в SPIR-V утилитой glslc при сборке.

Сцена и процедурные нормали

Сцена состоит из четырёх параллелепипедов, каждый строится генератором из лабораторной работы №1 по длинам рёбер a, b, c и углам α, β, γ между ними: параллелепипед по варианту (2 × 1,2 × 0,8, прямоугольный), пол (20 × 0,2 × 20), наклонный параллелепипед (α = 70°, β = 110°, γ = 60°) и колонна (0,7 × 3 × 0,7). Форму, положение, поворот, растяжение, скорость вращения вокруг вертикальной оси и материал любого объекта можно изменить в разделе «Объекты и материалы».

Каждый объект хранится как 24 вершины и 36 индексов: у каждой из шести граней собственные четыре вершины. Структура вершины `Vertex` содержит два атрибута - позицию (location = 0) и нормаль (location = 1), по три float, формат `VK_FORMAT_R32G32B32_SFLOAT`, шаг 24 байта. Атрибут цвета вершины из первой работы удалён: цвет поверхности теперь задаётся материалом объекта.

Нормали генерируются процедурно. Нормаль грани, натянутой на рёбра **u** и **w**, равна

$$\mathbf{n} = \frac{\mathbf{u} \times \mathbf{w}}{|\mathbf{u} \times \mathbf{w}|}.$$

Если нормаль направлена внутрь фигуры (её скалярное произведение с вектором из центра фигуры в центр грани отрицательно), векторы **u** и **w** меняются местами: это разворачивает нормаль наружу и делает обход вершин треугольников направленным против часовой стрелки при взгляде снаружи. Все четыре вершины грани получают одну и ту же нормаль, поэтому каждая грань освещается как плоская, что правильно для многогранника. Если бы угловая вершина была общей для трёх граней, нормали пришлось бы усреднять, и рёбра выглядели бы сглаженными.

Матрица модели собирается так же, как в первой работе, а для нормалей используется обратная транспонированная матрица её линейной части:

$$\mathbf{M} = \mathbf{T}(\mathbf{p}_0) \cdot \mathbf{R} \cdot \mathbf{S}(\boldsymbol{\sigma}), \qquad \mathbf{R} = \mathbf{R}_z\mathbf{R}_y\mathbf{R}_x \cdot \mathbf{R}_y(\omega t), \qquad \mathbf{N} = (\mathbf{RS})^{-\mathrm{T}} = \mathbf{RS}^{-1},$$

где $\mathbf{p}_0$ - позиция объекта, $\mathbf{R}_z\mathbf{R}_y\mathbf{R}_x$ - поворот углами Эйлера из интерфейса, $\mathbf{R}_y(\omega t)$ - вращение вокруг вертикальной оси с угловой скоростью ω, $\mathbf{S}(\boldsymbol{\sigma})$ - растяжение по осям. Так как $\mathbf{R}$ ортогональна, $(\mathbf{RS})^{-\mathrm{T}} = \mathbf{RS}^{-1}$, и при неравномерном растяжении нормали остаются перпендикулярными граням. Растяжение ограничено снизу значением 0,05, иначе матрица $\mathbf{N}$ вырождается.

Камера

Камера свободная: она задаётся позицией $\mathbf{e}$, азимутом θ (поворот вокруг вертикальной оси), наклоном ψ, вертикальным углом обзора φ, ближней и дальней плоскостями отсечения n и f. Направление взгляда вычисляется по двум углам функцией `directionFromAngles`:

$$\mathbf{f}(\theta, \psi) = (\sin\theta\cos\psi, \sin\psi, -\cos\theta\cos\psi).$$

При $\theta = \psi = 0$ камера смотрит вдоль оси −Z, при $\theta = 90°$ - вдоль +X. Этой же функцией задаются направления направленного источника и прожекторов, поэтому все направления в интерфейсе задаются одинаково - двумя углами. Матрица вида строится функцией lookAt, матрица перспективной проекции $\mathbf{P}_{\text{персп}}$ - так же, как в первой работе:

$$\mathbf{V} = \mathrm{lookAt}(\mathbf{e}, \mathbf{e} + \mathbf{f}, \mathbf{y}), \qquad \mathbf{y} = (0, 1, 0),$$

$$\mathbf{p}_{\text{мир}} = \mathbf{M} \cdot \mathbf{p}, \qquad \mathbf{p}_{\text{clip}} = \mathbf{P}_{\text{персп}} \cdot \mathbf{V} \cdot \mathbf{p}_{\text{мир}}.$$

Произведение $\mathbf{P}_{\text{персп}} \cdot \mathbf{V}$ общее для всех объектов, поэтому оно вычисляется на процессоре один раз за кадр и передаётся в шейдеры через uniform-буфер сцены, а матрица $\mathbf{M}$ - через uniform-буфер модели. Вершинный шейдер переводит вершину в мировые координаты, затем в пространство отсечения и передаёт мировые позицию и нормаль во фрагментный шейдер для расчёта освещения. Наклон ограничен диапазоном ±89°: при ±90° направление взгляда совпадает с вектором $\mathbf{y}$, их векторное произведение в lookAt обращается в ноль, и оси камеры не определены.

Кроме ползунков раздела «Камера», камерой можно управлять мышью и клавиатурой (функция `updateCameraInput`): при зажатой правой кнопке мыши смещение курсора меняет азимут и наклон (0,15° на пиксель),

клавиши W и S перемещают камеру вдоль **f**, A и D - вдоль горизонтального вектора $\mathbf{f} \times \mathbf{y}$, E и Q - вдоль **y**, колесо мыши меняет угол обзора. Перемещение за кадр вычисляется как

$$\mathbf{e} \leftarrow \mathbf{e} + v\Delta t \frac{\mathbf{m}}{|\mathbf{m}|},$$

где **m** - сумма векторов нажатых клавиш, v - скорость из интерфейса (с зажатым Shift она утраивается), Δt - время кадра, поэтому скорость движения не зависит от частоты кадров. Ввод не обрабатывается, когда мышь или клавиатура заняты интерфейсом (флаги ImGui `WantCaptureMouse` и `WantCaptureKeyboard`), например при вводе числа в поле.

Модель освещения Блинна-Фонга

Освещение рассчитывается во фрагментном шейдере в мировых координатах. Пусть **n** - нормаль (после интерполяции она нормируется заново), **l** - единичный вектор направления на источник, **v** - единичный вектор направления на камеру. Вклад одного источника по модели Блинна-Фонга:

$$\mathbf{c}_{\text{ист}} = (\mathbf{k}_d \max(\mathbf{n} \cdot \mathbf{l}, 0) + \mathbf{k}_s \max(\mathbf{n} \cdot \mathbf{h}, 0)^s) \odot \mathbf{L},$$

$$\mathbf{h} = \frac{\mathbf{l} + \mathbf{v}}{|\mathbf{l} + \mathbf{v}|}, \qquad \mathbf{v} = \frac{\mathbf{e} - \mathbf{p}}{|\mathbf{e} - \mathbf{p}|},$$

где **h** - вектор, делящий пополам угол между **l** и **v**, $\mathbf{k}_d$ - альбедо (цвет рассеянного отражения), $\mathbf{k}_s$ - цвет блика, s - коэффициент блика, **L** - освещённость от источника (цвет, умноженный на интенсивность и затухание), $\odot$ - покомпонентное умножение. В модели Фонга вместо $\mathbf{n} \cdot \mathbf{h}$ используется скалярное произведение отражённого луча и **v**; модель Блинна-Фонга даёт похожий блик, но он не обрывается резко при больших углах между **l** и **v**: пока оба вектора лежат с внешней стороны поверхности, $\mathbf{n} \cdot \mathbf{h} > 0$. Блик учитывается только при $\mathbf{n} \cdot \mathbf{l} > 0$, чтобы на гранях, отвёрнутых от источника, не появлялось бликов. Коэффициент блика ограничен снизу единицей, так как pow(0, 0) в GLSL не определён.

Итоговый цвет фрагмента складывается из фонового света и вкладов всех включённых источников:

$$\mathbf{c} = \mathbf{k}_d \odot \mathbf{c}_\text{ф} I_\text{ф} + \sum \mathbf{c}_{\text{ист}},$$

где $\mathbf{c}_\text{ф}$ и $I_\text{ф}$ - цвет и интенсивность фонового света. Для разных типов источников различаются только $\mathbf{l}$ и $\mathbf{L}$ (таблица 1); $\mathbf{c}_L$ и I_L - цвет и интенсивность источника.

Тип источника	Направление на источник	Освещённость
Направленный	$-\mathbf{d}_\text{н}$	$\mathbf{c}_L I_L$
Точечный	$\dfrac{\mathbf{p}_L - \mathbf{p}}{r}$	$\dfrac{\mathbf{c}_L I_L}{r^2}$
Прожектор	$\dfrac{\mathbf{p}_L - \mathbf{p}}{r}$	$\kappa \dfrac{\mathbf{c}_L I_L}{r^2}$

Таблица 1 — Направление на источник и освещённость для разных типов источников

Здесь $\mathbf{d}_\text{н}$ - направление, в котором светит направленный источник, $\mathbf{p}_L$ - позиция источника, $r = |\mathbf{p}_L - \mathbf{p}|$ - расстояние до него. Направленный источник считается бесконечно удалённым и не затухает. У точечного источника и прожектора освещённость убывает по закону обратных квадратов: при удвоении расстояния она уменьшается в четыре раза. Чтобы не делить на ноль, квадрат расстояния в шейдере ограничен снизу значением 10^{-4}.

Мягкий край прожектора задаётся двумя углами, отсчитываемыми от оси конуса: внутренним $\varphi_\text{вн}$ и внешним $\varphi_\text{внеш}$. Множитель конуса κ вычисляется по косинусу угла γ между осью прожектора $\mathbf{d}_\text{пр}$ и направлением от прожектора на точку:

$$\cos\gamma = (-\mathbf{l}) \cdot \mathbf{d}_\text{пр}, \qquad \kappa = \operatorname{clamp}\left(\frac{\cos\gamma - \cos\varphi_\text{внеш}}{\cos\varphi_\text{вн} - \cos\varphi_\text{внеш}}, 0, 1\right).$$

Внутри внутреннего конуса $\kappa = 1$, за внешним $\kappa = 0$, между ними множитель линейно убывает по косинусу угла, поэтому граница светового пятна размыта. Если внешний угол равен внутреннему, знаменатель обращается в ноль; в этом случае шейдер использует ступенчатую функцию $\operatorname{step}(\cos\varphi_\text{внеш}, \cos\gamma)$, и край пятна получается резким. Косинусы углов вычисляются на процессоре, а интерфейс не позволяет сделать внешний угол меньше внутреннего. Любой прожектор можно прикрепить к камере флажком «Прикрепить к камере (фонарик)»: тогда его позиция и направление каждый кадр берутся у камеры.

Стартовый код через vk-bootstrap выбирает для swapchain формат с sRGB-кодированием (`VK_FORMAT_B8G8R8A8_SRGB`), поэтому освещение считается в линейном пространстве, а перевод в sRGB выполняется аппаратно при записи цвета.

Наборы дескрипторов

Внешние данные шейдеров разделены на два набора дескрипторов по частоте изменения (дополнительное задание 1): данные сцены общие для всех объектов и привязываются один раз за кадр, а данные модели у каждого объекта свои (таблица 2).

Набор	Binding	Тип дескриптора	Содержимое	Стадии
0	0	`UNIFORM_BUFFER`	`SceneData`: матрица P·V, позиция камеры, фоновый свет, направленный источник	вершинная, фрагментная
0	1	`STORAGE_BUFFER`	массив точечных источников	фрагментная
0	2	`STORAGE_BUFFER`	массив прожекторов	фрагментная
1	0	`UNIFORM_BUFFER`	`ModelData`: матрица модели, матрица нормалей, материал	вершинная, фрагментная

Таблица 2 — Наборы дескрипторов

Создаются две раскладки `VkDescriptorSetLayout`. Раскладка конвейера `VkPipelineLayout` получает их в порядке «сцена, модель», и этот порядок задаёт номера set = 0 и set = 1 в шейдерах. Пул дескрипторов рассчитан на 1 + N наборов (N - число объектов), 1 + N uniform-дескрипторов и два storage-дескриптора. Наборы моделей выделяются одним вызовом `vkAllocateDescriptorSets`, и каждый связывается с uniform-буфером своего объекта через `vkUpdateDescriptorSets`.

При отрисовке набор сцены привязывается один раз (`vkCmdBindDescriptorSets` с firstSet = 0), а для каждого объекта привязываются только его набор модели (firstSet = 1), вершинный и индексный буферы, после чего вызывается `vkCmdDrawIndexed`. Набор 0 при этом остаётся привязанным, поскольку раскладка конвейера не меняется.

Storage-буферы с источниками

Точечные источники и прожекторы (дополнительные задания 2 и 3) хранятся в двух storage-буферах. Массив в шейдере объявлен без размера, а количество источников записано в начале самого буфера:

```
layout(set = 0, binding = 1, std430)
readonly buffer PointLightBuffer {
    uint count;
    PointLight lights[];
} point_lights;
```

Поэтому в шейдере нет константы с максимальным числом источников, а цикл идёт от 0 до count. Для storage-буферов используется раскладка std430. Структуры источников выравниваются по 16 байт (как

vec3), поэтому массив начинается со смещения 16, а за каждым полем vec3 следует поле float - полезный параметр или добивка (таблица 3). Структуры `PointLightData`, `SpotLightData` и `LightBufferHeader` в C++ повторяют эту раскладку.

Поле	Тип GLSL	Смещение, байт
Начало буфера		
`count`	uint	0
`lights[]`	PointLight / SpotLight	16
PointLight (точечный источник), размер и шаг массива 32 байта		
`position`	vec3	0
`intensity`	float	12
`color`	vec3	16
`padding`	float	28
SpotLight (прожектор), размер и шаг массива 48 байт		
`position`	vec3	0
`intensity`	float	12
`direction`	vec3	16
`cos_inner`	float	28
`color`	vec3	32
`cos_outer`	float	44

Таблица 3 — Раскладка storage-буферов с источниками (std430)

После запуска каждый буфер рассчитан на 8 источников. Каждый кадр функция `uploadLights` записывает в буфер количество включённых источников и их массив; выключенные источники в буфер не попадают. Если источников больше, чем помещается, создаётся новый буфер ёмкостью

$$C_{\text{нов}} = \max(K, 2C),$$

где K - число источников, C - прежняя ёмкость. Старый буфер удаляется, а дескриптор в наборе сцены перенаправляется на новый буфер через `vkUpdateDescriptorSets`. Это выполняется в функции `render()`, то есть после того, как `prepare()` дождался fence предыдущего кадра: в этот момент GPU уже не читает старый буфер, и набор дескрипторов не используется ни одним командным буфером, ожидающим выполнения. Удвоение ёмкости нужно, чтобы при добавлении источников по одному буфер пересоздавался редко (8 → 16 → 32 …). Над списками источников в

интерфейсе выводятся число включённых источников и текущая ёмкость буфера.

Визуализация источников

Источники света показываются на сцене (дополнительное задание 4). Каждый включённый точечный источник и прожектор рисуется кубиком с ребром 0,2 цвета самого источника, а у прожектора отрезками рисуются ось и конус: белые образующие и окружность внешнего угла и серая окружность внутреннего угла. Флажки «Показывать кубики источников» и «Показывать конусы прожекторов» в разделе «4. Визуализация источников» скрывают и показывают фигуры, ползунок «Длина конуса» задаёт длину образующих L.

Кубик строится один раз при запуске тем же генератором, что и параллелепипеды сцены, и его 36 индексов разворачиваются в список вершин треугольников. Каждый кадр функция `buildLightMarkers` формирует массив вершин `MarkerVertex` (позиция в мировых координатах и цвет, 24 байта): сначала копии кубика, сдвинутые в позиции источников, затем отрезки конусов. Точки конуса вычисляются по оси прожектора $\mathbf{d}_{\text{пр}}$ и двум перпендикулярным ей единичным векторам $\mathbf{q}_1$ и $\mathbf{q}_2$:

$$\mathbf{q}_1 = \frac{\mathbf{d}_{\text{пр}} \times \mathbf{g}}{\left|\mathbf{d}_{\text{пр}} \times \mathbf{g}\right|}, \qquad \mathbf{q}_2 = \mathbf{d}_{\text{пр}} \times \mathbf{q}_1,$$

$$\mathbf{x}(\varphi, \tau) = \mathbf{p}_L + L\big(\mathbf{d}_{\text{пр}}\cos\varphi + (\mathbf{q}_1\cos\tau + \mathbf{q}_2\sin\tau)\sin\varphi\big),$$

где $\mathbf{g} = (0, 1, 0)$, а если ось почти вертикальна ($\left|d_y\right| \geq 0{,}99$), то $\mathbf{g} = (1, 0, 0)$, чтобы векторное произведение не обратилось в ноль; φ - внешний или внутренний угол, τ - угол поворота вокруг оси. Все образующие имеют одинаковую длину L, поэтому даже конус с углом, близким к 90°, остаётся конечным. Конус состоит из оси, восьми образующих и двух окружностей по 32 отрезка. Выключенные источники и прожектор, прикреплённый к камере, не рисуются: камера находится внутри его кубика.

Вершины записываются в отдельный вершинный буфер функцией `uploadMarkers`. Число вершин меняется вместе с числом источников, поэтому буфер пересоздаётся по тому же правилу $C_{\text{нов}} = \max(K, 2C)$, но дескриптор обновлять не нужно: вершинный буфер привязывается командой `vkCmdBindVertexBuffers` при записи командного буфера.

Фигуры рисуются после объектов сцены двумя дополнительными конвейерами с шейдерами `light_marker.vert` и `light_marker.frag`:

кубики - с топологией `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST` и отсечением задних граней, конусы - с топологией `VK_PRIMITIVE_TOPOLOGY_LINE_LIST` без отсечения (у отрезка нет сторон) и толщиной линии 1 (другая толщина требует возможности устройства wideLines). Все три конвейера создаются одной функцией `createGraphicsPipeline`, которой передаются шейдеры, описание вершины, топология и режим отсечения. Вершинный шейдер визуализации только умножает позицию на матрицу $\mathbf{P}_{\text{персп}} \cdot \mathbf{V}$ из uniform-буфера сцены, а фрагментный выводит цвет вершины без изменений, поэтому фигуры имеют постоянный цвет и не освещаются по модели Блинна-Фонга. Раскладка у конвейеров общая, поэтому набор сцены, привязанный в начале кадра, остаётся действующим, а два вызова `vkCmdDraw` для треугольников и отрезков отличаются только числом вершин и смещением firstVertex. Тест глубины включён, поэтому объекты сцены закрывают фигуры источников.

Раскладка uniform-буферов

Uniform-блоки объявлены с раскладкой std140, и структуры `SceneData` и `ModelData` в C++ повторяют их байт в байт: смещения проверяются `static_assert` и дополнительно были сверены с дизассемблированным SPIR-V. В std140 поле vec3 начинается с адреса, кратного 16, и занимает 12 байт, поэтому после каждого vec3 стоит float (таблица 4).

Поле	Тип GLSL	Смещение, байт	Размер, байт
SceneData (набор 0, binding 0)			
`view_projection`	mat4	0	64
`camera_position`	vec3	64	12
`padding0`	float	76	4
`ambient_color`	vec3	80	12
`ambient_intensity`	float	92	4
`light_direction`	vec3	96	12
`light_intensity`	float	108	4
`light_color`	vec3	112	12
`padding1`	float	124	4
Итого			128
ModelData (набор 1, binding 0)			
`transform`	mat4	0	64
`normal_matrix`	mat4	64	64

Поле	Тип GLSL	Смещение, байт	Размер, байт
`albedo`	vec3	128	12
`shininess`	float	140	4
`specular_color`	vec3	144	12
`padding`	float	156	4
Итого			160

Таблица 4 — Раскладка uniform-буферов (std140)

Данные для всех буферов вычисляются в `update()`, а записываются в отображённую память в `render()`: в стартовом коде один кадр «в полёте», и только после ожидания fence гарантировано, что GPU закончил читать данные предыдущего кадра. Ползунки, от которых зависит корректность вычислений (наклон камеры, углы параллелепипеда, коэффициент блика, углы прожектора), ограничены флагом `ImGuiSliderFlags_AlwaysClamp`, чтобы значение нельзя было вывести за допустимый диапазон вводом с клавиатуры. Исправление пула дескрипторов ImGui в стартовом коде и загрузка системного шрифта Segoe UI для русских подписей перенесены из первой работы.

Результаты

Слева в окне программы находится панель ImGui с разделами «Камера», «Фоновый свет», «Направленный источник», «Объекты и материалы», «2. Точечные источники», «3. Прожекторы» и «4. Визуализация источников»; номера разделов соответствуют дополнительным заданиям. Сцена отрисовывается в реальном времени, все параметры применяются сразу. Программа проверялась со слоем валидации Vulkan: ошибок и предупреждений не было, в том числе при добавлении и удалении источников и пересоздании storage-буферов.

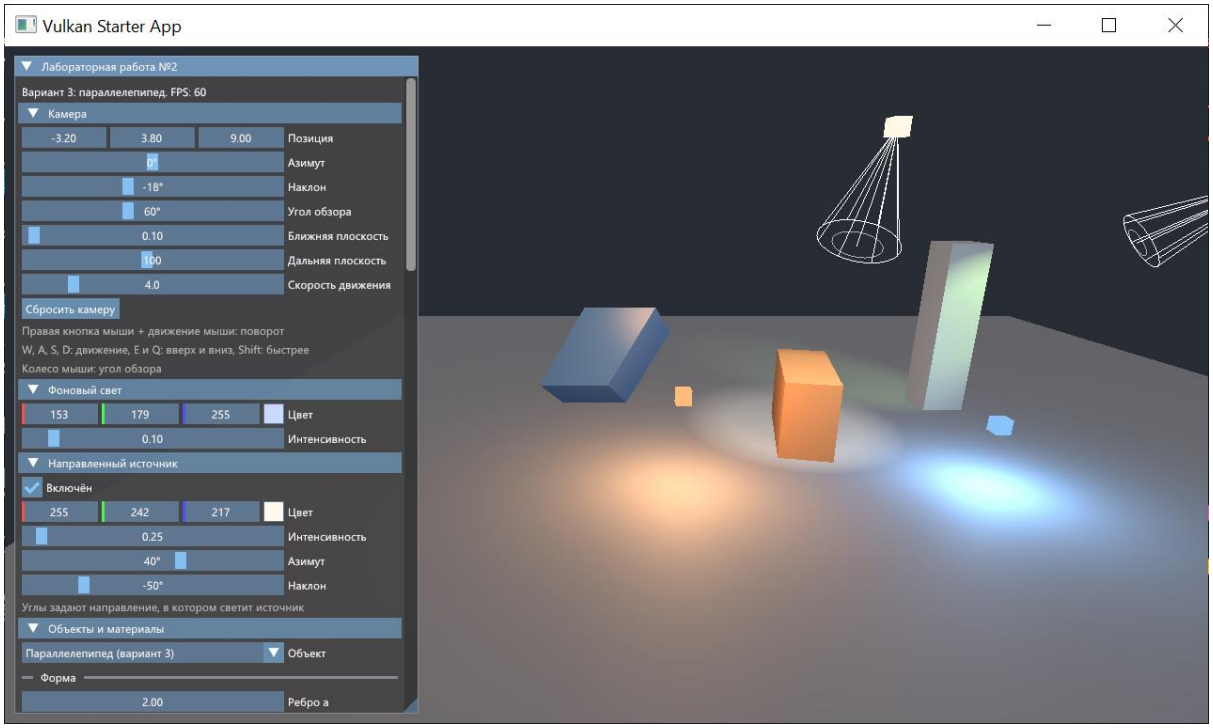


Рисунок 1 - Сцена при запуске программы

На рисунке 1 показана сцена при запуске. Параллелепипед по варианту вращается вокруг вертикальной оси, рядом находятся наклонный параллелепипед и колонна, все объекты стоят на полу. Сцену освещают слабый голубоватый фоновый свет, направленный источник, два точечных источника (оранжевый слева и синий справа) и два прожектора: белый светит сверху на параллелепипед и даёт на полу пятно с мягким краем, зелёный освещает колонну. Каждый источник показан кубиком своего цвета, у прожекторов белыми линиями нарисованы ось и конус. Освещённость пола вокруг точечных источников быстро убывает с расстоянием по закону обратных квадратов.

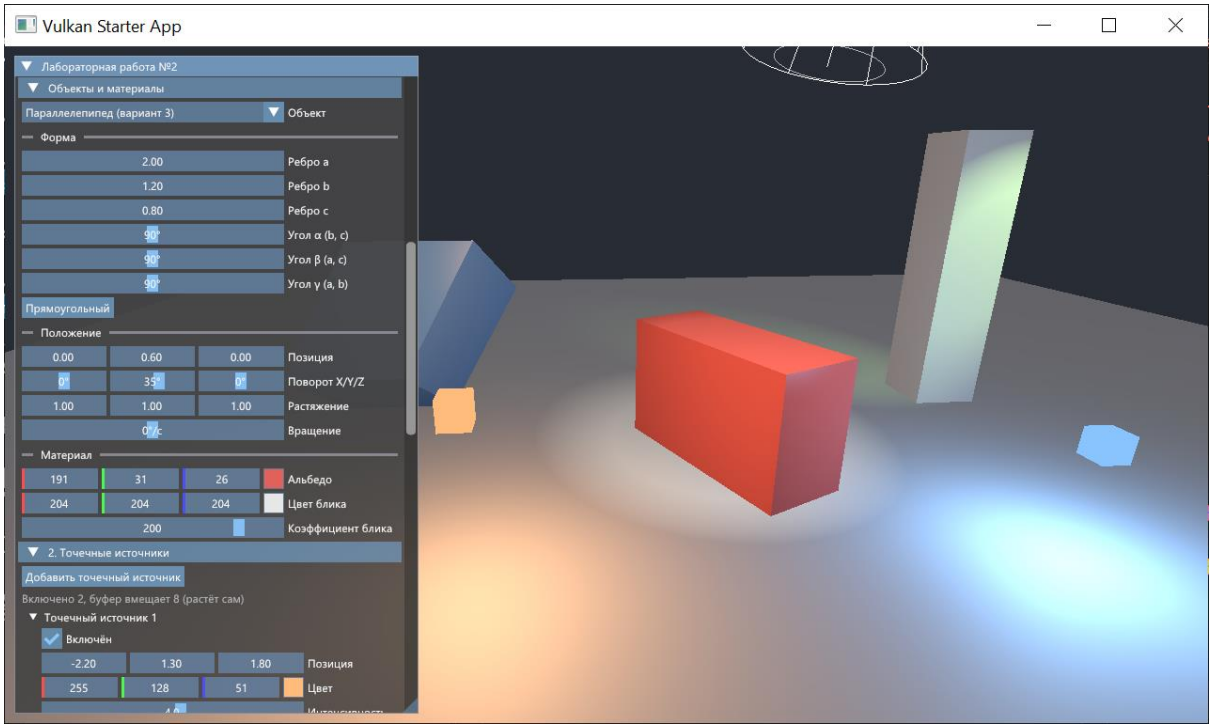


Рисунок 2 - Изменение положения и материала объекта

На рисунке 2 камера приближена к параллелепипеду. В разделе «Объекты и материалы» объект повёрнут на 35° вокруг оси Y, вращение остановлено, альбедо заменено на красное, а коэффициент блика увеличен до 200. Грани, по-разному повёрнутые к источникам, освещены по-разному; ярче всего верхняя грань, на которую светит прожектор.

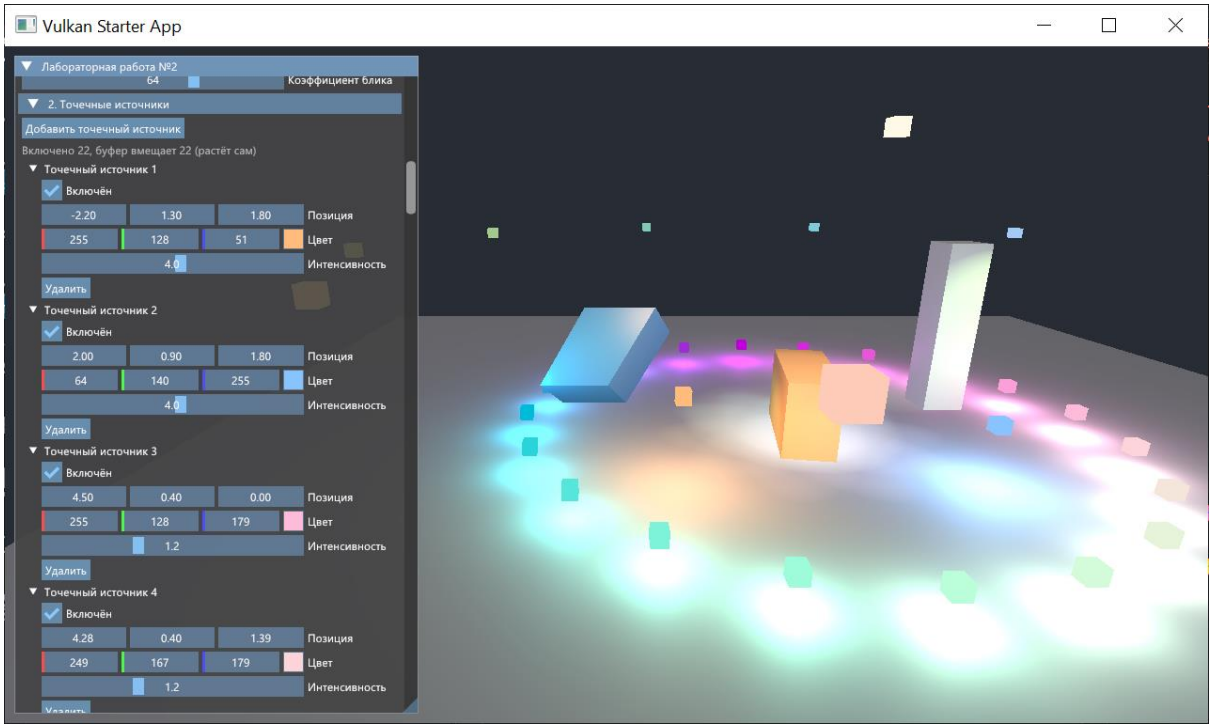


Рисунок 3 - Сцена с большим количеством точечных источников и прожекторов

На рисунке 3 добавлено больше восьми точечных источников и прожекторов, расставленных по кругу. Надпись над списком показывает, что источников стало больше начальной ёмкости буфера и буфер был автоматически пересоздан большего размера; все источники учитываются шейдером, хотя размер массива в нём не задан. Кубики показывают положение и цвет каждого источника, а конусы прожекторов здесь скрыты флажком «Показывать конусы прожекторов». Каждый источник можно выключить флажком «Включён» или удалить кнопкой «Удалить».

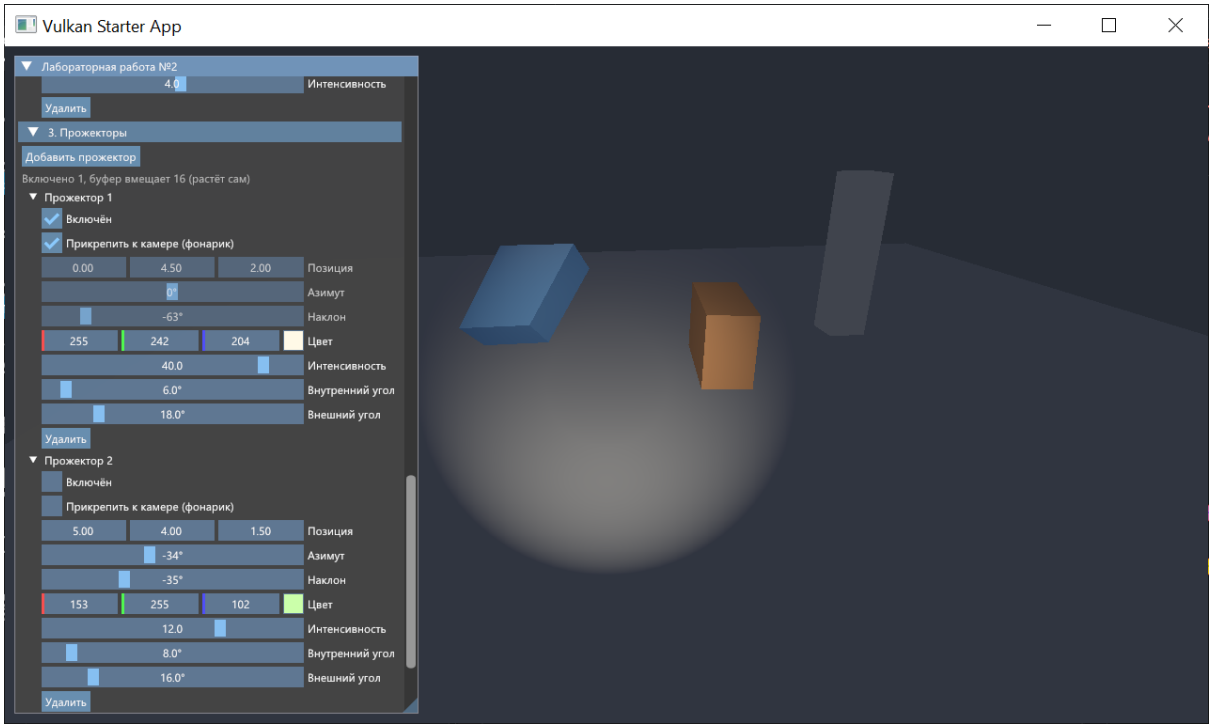


Рисунок 4 - Прожектор, прикреплённый к камере

На рисунке 4 направленный источник и остальные источники выключены, первый прожектор прикреплён к камере, его внутренний угол 6°, внешний 18°. Пятно света находится в центре кадра и перемещается вместе с камерой, поля позиции и направления прожектора в интерфейсе неактивны, а его кубик и конус не рисуются. Между внутренним и внешним конусом освещённость плавно убывает до нуля; если сделать углы равными, край пятна становится резким.

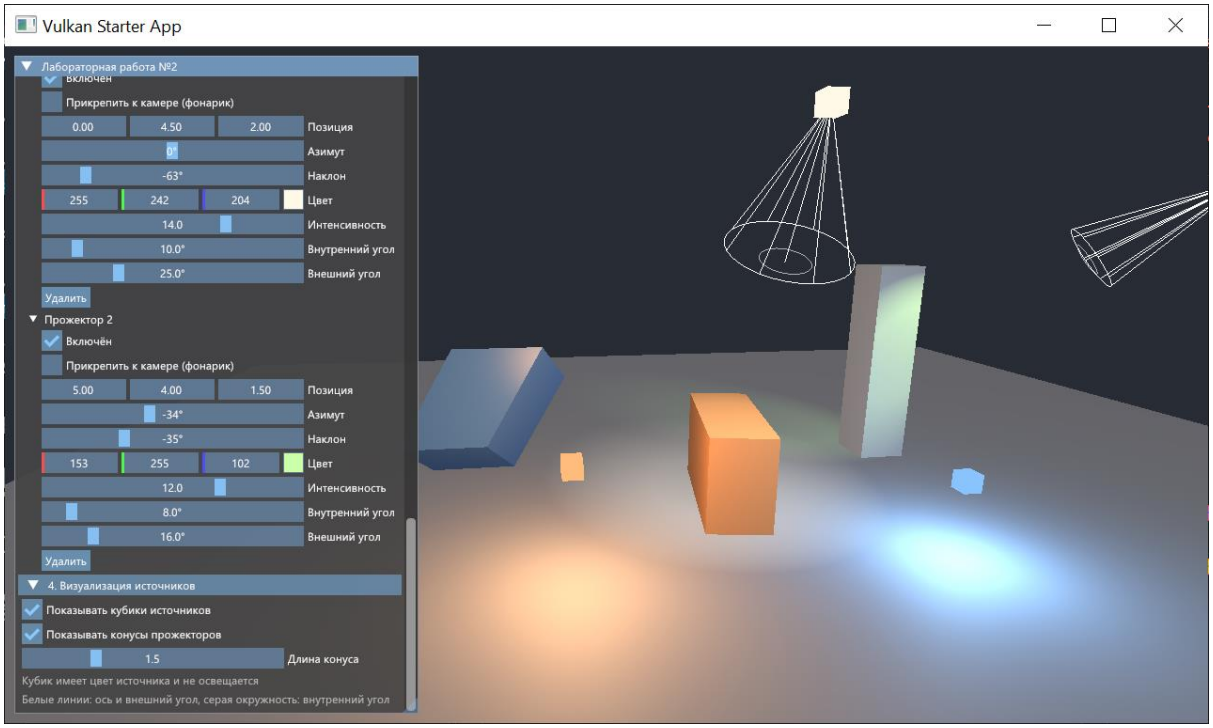


Рисунок 5 - Визуализация источников света

На рисунке 5 камера приближена к источникам, внизу панели открыт раздел «4. Визуализация источников». У первого прожектора внутренний угол 10°, внешний 25°: белые образующие и окружность показывают внешний конус, серая окружность - внутренний, белая линия в центре - ось прожектора; справа виден конус второго прожектора, направленного на колонну. Кубики точечных источников окрашены в их цвета, оранжевый и синий, и, в отличие от объектов сцены, все их грани одинаково яркие, так как фигуры не освещаются. Флажки раздела скрывают кубики и конусы, ползунок «Длина конуса» меняет длину образующих.

Выводы

В ходе работы я научилась реализовывать свободную камеру: задавать направление взгляда двумя углами, строить по нему матрицу вида и управлять камерой как из интерфейса, так и мышью и клавиатурой с перемещением, не зависящим от частоты кадров. Я разобралась в модели освещения Блинна-Фонга - из чего складываются фоновая, диффузная и зеркальная составляющие, зачем нужен вектор, делящий пополам угол между направлениями на источник и на камеру, и почему блик учитывается только на освещённой стороне, - и реализовала три типа источников: направленный, точечный и прожектор с затуханием по закону обратных квадратов и мягким краем.

Я научилась разделять данные шейдеров на наборы дескрипторов по частоте их изменения, использовать storage-буферы с раскладкой std430 и массивами без заданного размера, а также безопасно пересоздавать буферы и обновлять дескрипторы, когда GPU закончил предыдущий кадр. Кроме того, я закрепила правила выравнивания std140 и std430 для полей vec3 и научилась проверять раскладку структур с помощью static_assert и дизассемблированного SPIR-V. Для визуализации источников я научилась создавать несколько графических конвейеров с общей раскладкой, рисовать отрезки с топологией LINE_LIST и выводить вспомогательную геометрию без освещения, формируя её каждый кадр в динамическом вершинном буфере.

Эксперименты со сценой показали, что затухание по закону обратных квадратов делает освещённость очень неравномерной: поверхность рядом с источником намного ярче удалённой, поэтому интенсивность точечных источников и прожекторов приходится задавать в десятки раз больше, чем у направленного. Мягкий край прожектора выглядит естественнее резкого, а при увеличении коэффициента блика блик становится меньше и резче, и поверхность выглядит более гладкой.